MarineMind: AI-Powered Solutions for Modern Shipping

Sara Batatawala
Dept. of Artificial Intelligence and Data Science Engineering, Rizvi College of Engineering, Mumbai, India
sarabatatawala@eng.rizvi.edu.in

Mahek Shaikh
Dept. of Artificial Intelligence and Data Science Engineering, Rizvi College of Engineering, Mumbai, India
mahekshaikh@eng.rizvi.edu.in

Sameer Ansari
Dept. of Artificial Intelligence and Data Science Engineering, Rizvi College of Engineering, Mumbai, India
sameeransari@eng.rizvi.edu.in

Prof. Nargis Shaikh
Dept. of Artificial Intelligence and Data Science Engineering, Rizvi College of Engineering, Mumbai, India
hod.ecs@eng.rizvi.edu.in

Prof. Junaid Mandviwala
Dept. of Artificial Intelligence and Data Science Engineering, Rizvi College of Engineering, Mumbai, India
junaid@eng.rizvi.edu.in

Abstract*—The maritime industry generates vast amounts of operational data through daily noon reports, technical manuals, and regulatory documents, yet most vessel operators still rely on manual analysis and fragmented information systems for performance monitoring and troubleshooting. This paper presents MarineMind, a multi-agent AI system designed to unify maritime knowledge retrieval, vessel performance analytics, and intelligent machinery diagnostics into a single conversational platform. The system employs a five-agent architecture comprising a Guardrails Agent for query safety filtering, a Router Agent for intent classification, a Retrieval-Augmented Generation (RAG) Agent with a five-stage pipeline (query rewriting, query expansion, multi-query vector retrieval, cross-encoder reranking, and grounded generation), a Graph Agent for automated chart generation from noon report data, and a Diagnosis Agent that performs hybrid reasoning by combining real-time operational data analysis with technical knowledge retrieval. The backend is built on Django with Pinecone as the vector database, SentenceTransformer for embeddings, and HuggingFace-hosted large language models for natural language understanding and generation. The React-based frontend renders interactive Chart.js visualizations, citation-linked answers, structured diagnostic reports, and real-time alert notifications for operational anomalies. Evaluation across document retrieval accuracy, graph generation correctness, diagnostic reasoning quality, and end-to-end response latency demonstrates that the multi-agent approach outperforms monolithic chatbot architectures in domain-specific maritime scenarios by achieving higher retrieval precision through multi-query expansion and cross-encoder reranking, while maintaining sub-two-minute response times for complex diagnostic queries.

Keywords*—maritime industry, artificial intelligence, retrieval-augmented generation, ship maintenance, cognitive assistant.

I. Introduction

The global shipping industry is responsible for approximately 90% of world trade, operating over 100,000 commercial vessels that generate enormous volumes of operational data daily. Each vessel produces noon reports containing 50+ performance parameters including fuel consumption, engine RPM, speed, weather conditions, draft readings, and cargo status. Additionally, vessels carry hundreds of pages of technical manuals, standard operating procedures (SOPs), and maintenance guides. Despite this data richness, ship operators and marine engineers face persistent challenges: operational data is locked in spreadsheets with no analytical tooling, technical knowledge is buried in PDF manuals with no search capability, and machinery troubleshooting relies heavily on individual experience rather than systematic data-driven diagnosis. Recent advances in Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) have demonstrated strong potential for domain-specific question answering. However, applying these techniques to the maritime domain presents unique challenges. First, maritime queries span multiple modalities — from factual lookups in manuals ("What is the turbocharger maintenance schedule?") to analytical questions requiring data visualization ("Show fuel consumption trend for the last 30 days") to diagnostic reasoning ("Why is engine exhaust temperature abnormally high?"). A single-agent RAG pipeline cannot adequately handle this diversity. Second, maritime technical documents contain specialized terminology, tabular data, and safety-critical information that demands high retrieval precision and factual grounding. Third, machinery diagnosis requires combining real-time operational data with knowledge from maintenance manuals — a hybrid reasoning task that no single retrieval approach can solve. This paper presents MarineMind, a multi-agent AI platform that addresses these challenges through a modular agent architecture. The system decomposes complex maritime queries through a pipeline of specialized agents: the Guardrails Agent ensures query safety and domain relevance, the Router Agent classifies queries into five distinct categories (RAG, Internet search, hybrid, graph analytics, and diagnosis), and specialized execution agents handle each category with tailored pipelines. The RAG Agent implements a five-stage advanced retrieval pipeline with query rewriting, multi-query expansion, vector retrieval from Pinecone, cross-encoder reranking using MS-MARCO MiniLM, and citation-grounded generation. The Graph Agent parses natural language into structured intents, resolves vessel references via fuzzy matching, fetches analytical data, and

produces Chart.js-compatible visualizations. The Diagnosis Agent performs hybrid reasoning by combining statistical anomaly detection from noon report data with knowledge retrieval from technical manuals.

The key contributions of this paper are:

- A multi-agent architecture specifically designed for the maritime domain that routes queries to specialized agents based on intent classification.
- An advanced five-stage RAG pipeline with multi-query expansion and cross-encoder reranking that improves retrieval precision for maritime technical documents.
- A hybrid diagnosis agent that combines operational data analytics with document-based knowledge retrieval for intelligent machinery troubleshooting.
- A natural-language-to-chart pipeline that enables non-technical users to visualize vessel performance data through conversational queries.
- A fully integrated web platform with document ingestion, noon report import, fleet management, a rule-based alert engine for operational anomaly detection, and an administrative dashboard.

II. Problem Statement

The maritime industry faces several critical challenges in knowledge management and operational decision-making that current systems fail to address holistically:

A. Fragmented Knowledge Access

Marine engineers and vessel operators must consult multiple disconnected sources to find information — PDF manuals for technical procedures, Excel spreadsheets for performance data, email chains for operational updates, and regulatory databases for compliance requirements. There is no unified interface that allows natural language querying across these diverse data sources. Existing maritime software systems treat document management, performance analytics, and communication as separate silos, forcing users to context-switch between tools.

B. Lack of Intelligent Data Analysis

Vessels generate daily noon reports containing 50+ parameters covering engine performance, fuel consumption, speed, weather conditions, draft, and cargo status. Despite this data richness, most operators lack the technical expertise or tooling to perform meaningful analytics. Identifying trends in fuel consumption, detecting anomalies in engine performance, or understanding the impact of weather on vessel efficiency requires statistical analysis that is beyond manual spreadsheet inspection. The absence of automated visualization tools means that actionable insights remain buried in raw data.

C. Reactive Machinery Troubleshooting

When machinery problems arise at sea, engineers typically rely on personal experience and manual lookup of relevant procedures. This reactive approach has significant drawbacks: diagnosis quality varies with individual expertise, relevant information may span multiple manual sections, and there is no systematic way to correlate observed symptoms with historical operational data. For example, diagnosing the root cause of elevated exhaust temperatures requires simultaneously checking fuel quality records, engine load trends, turbocharger performance data, and cooling system maintenance history — information scattered across different systems.

D. Limitations of General-Purpose AI Systems

While general-purpose LLM chatbots (e.g., ChatGPT, Gemini) can answer broad questions, they lack access to vessel-specific documentation, operational data, and internal procedures. They cannot query a ship's noon report database, retrieve from uploaded technical manuals, or perform data-driven diagnosis. Furthermore, they are prone to hallucination in specialized maritime contexts where accuracy is safety-critical. There is a need for a domain-specific AI system that combines the conversational fluency of LLMs with grounded retrieval from authoritative sources and access to real-time operational data.

E. Research Gap

Existing literature on maritime AI focuses primarily on autonomous navigation, route optimization, and predictive maintenance using sensor data. Few studies explore the integration of RAG-based knowledge retrieval with operational analytics in a unified conversational interface for marine engineering support. The multi-agent paradigm, while explored in general AI contexts, has not been systematically applied to the maritime operations domain where query types span from factual lookups to analytical visualization to diagnostic reasoning.

III. Literature Review

A. Deep Learning in Maritime Anomaly Detection

Kuhl et al. [1] proposed an anomaly detection system for vessels using LSTM Autoencoders coupled with interpretable Decision Trees. The framework improved model transparency but lacked adaptability across diverse vessel conditions.

B. AIS-Based Outlier Detection

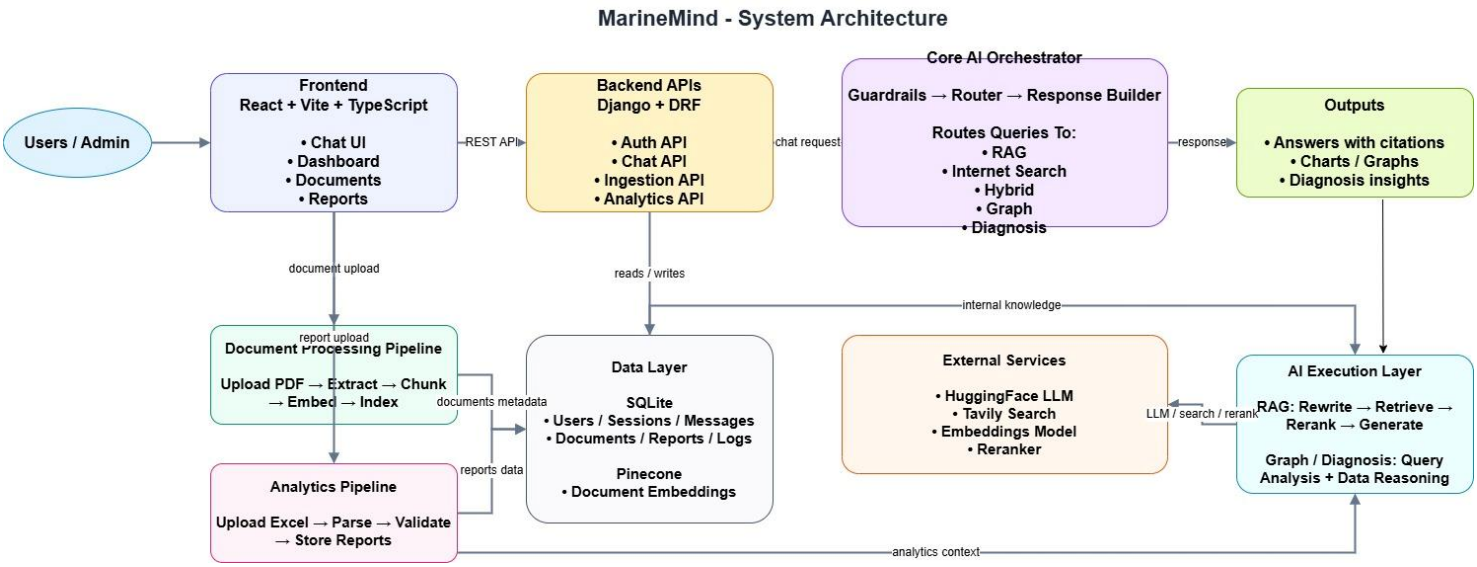
Chen et al. [2] developed a Bi-GRU Encoder–Decoder model for trajectory-based outlier detection using AIS data. The method effectively detected navigational anomalies but ignored onboard equipment behavior and textual documentation.

C. *Graph Neural Networks for Vessel Behavior Analysis*

Luo et al. [3] introduced an Adaptive Sparsified Graph Learning approach for multi-ship anomaly detection. Although robust for pattern recognition, it relied on synthetic datasets, limiting real-world applicability.

D. Identified Research Gap

Existing maritime research mainly focuses on navigation and sensors, overlooking crucial machinery manuals. MarineMind uses a multimodal RAG framework to interpret text and diagrams. It provides intelligent, context-aware, and explainable support for ship engineers.



IV. Proposed System

A. *System Architecture*

MarineMind follows a three-tier architecture comprising a React-based frontend, a Django REST backend, and external AI services.

- Frontend Layer:
  Built with React 19, TypeScript, and Vite, the frontend provides five primary interfaces — ChatPage (conversational AI), DashboardPage (system monitoring), DocumentsPage (PDF ingestion), VesselsPage (fleet management), and NoonReportsPage (operational data). Chat responses are rendered with react-markdown for formatted text, Chart.js via react-chartjs-2 for interactive graphs, and structured diagnostic cards for machinery diagnosis results. Session-based navigation is managed through react-router-dom, and real-time feedback is provided via react-hot-toast.

- Backend Layer:
  The Django backend (v5.0+) with Django REST Framework exposes RESTful APIs for authentication, chat processing, document ingestion, analytics, and administration. The core AI pipeline is implemented as a modular agent system in the `agents/` package. Operational data is stored in SQLite with models for Vessels, NoonReports, ChatSessions, ChatMessages, Documents, SystemLogs, and AuditLogs. The administration module provides role-based access control (admin, engineer, analyst, viewer) through a customized Django admin interface.

- External Services Layer:
  The system integrates three external services: Pinecone (serverless vector database on AWS us-east-1 with cosine similarity), HuggingFace Inference API (OpenAI-compatible chat completions endpoint for LLM calls), and Tavily API (web search for real-time external information)

B. *Multi-Agent Pipeline*

Every user query passes through a three-step orchestration pipeline:

- Guardrails Agent : The first layer performs safety filtering and domain relevance checking. Using a zero-temperature LLM call, it classifies queries into three categories: "greeting" (responded to directly), "reject" (harmful, off-topic, or prompt injection attempts are blocked), or "pass" (valid maritime/technical queries proceed). This ensures the system rejects inappropriate content while being liberal with technically relevant queries.

- Router Agent: The router performs deterministic query classification using a two-phase approach: keyword-based pre-classification for speed (detecting terms like "show trend", "plot", "chart", "graph" for graph routing, or symptom-related terms for diagnosis), followed by LLM-based classification for ambiguous queries. Queries are routed to exactly one of five categories: "rag" (internal document retrieval), "internet" (external web search), "hybrid" (combined RAG + internet), "graph" (data visualization), or "diagnosis" (machinery troubleshooting).

- Agent Execution: Based on the routing decision, one of five specialized agents executes:

- RAG Agent (Five-Stage Pipeline)

  The RAG Agent implements an advanced retrieval pipeline optimized for maritime technical documents:
  Stage 1 — Query Rewriting:The user's raw query reformulated by the LLM to improve retrieval effectiveness. For example, "how to fix boiler?" becomes "boiler maintenance and troubleshooting procedures for marine auxiliary boilers."

Stage 2 — Query Expansion: The rewritten query is expanded into 3–4 semantically related search variations to improve recall. This addresses vocabulary mismatch between user queries and technical document language.
Stage 3 — Multi-Query Retrieval: Each query variant (rewritten + expanded) is embedded using Sentence Transformer and searched against the Pinecone vector index with top_k=8 per query. Results are deduplicated by text content, retaining the highest retrieval score for each unique chunk.
Stage 4 — Cross-Encoder Reranking: All candidate chunks are reranked against the original user query using a cross-encoder model (cross-encoder/ms-marco-MiniLM-L-6-v2). Chunks below a relevance threshold of 0.1 are filtered, and the top 5 are retained. This significantly improves precision over embedding-only retrieval.
Stage 5 — Grounded Generation:The LLM generates a response using only the reranked chunks as context, with mandatory inline citations ([doc1], [doc2], etc.) linking to source documents. Citation maps are resolved to actual document file URLs stored in SQLite.

- Search Agent (Internet)
  For queries requiring real-time external information (e.g., latest IMO regulations, current fuel prices), the Search Agent queries the Tavily API with a maritime-scoped search prefix. Top 5 web results are formatted with citation labels ([web1], [web2]) and the LLM generates a response grounded in these sources.

- Hybrid Agent
  When a query requires both internal documents and external data, the Hybrid Agent runs the RAG Agent and Search Agent in parallel, then merges their results. Citation maps from both sources are combined, and the LLM produces a unified response synthesizing internal documentation with external information.

- Graph Agent
  The Graph Agent converts natural language analytical queries into interactive visualizations through a six-step pipeline:

i. Intent Parsing (LLM): Extract structured parameters graph type, metric, vessel name/IMO, time range, axis labels, and title.
ii. Vessel Resolution: Fuzzy-match the vessel name or IMO number against the database. If only one vessel exists, it defaults to that vessel.
iii. Date Range Resolution: Convert natural language time ranges ("last 30 days", "past 3 months") into concrete date boundaries.
iv. Data Fetching: Query the analytics engine which computes aggregate metrics from NoonReport data using Django ORM aggregations (Avg, Sum, Min, Max, Count, StdDev, Trunc). Supported analyses include fuel consumption trends, speed vs. consumption scatter, RPM performance, voyage comparisons, weather impact analysis, anomaly detection (>2 standard deviations), fleet-wide comparisons, and vessel summaries.
v. Chart Configuration: Transform analytics data into Chart.js-compatible JSON configurations supporting line charts, bar charts, scatter plots, summary cards, and data tables.
vi. Interpretation (LLM): Generate 2–3 sentence analytical commentary highlighting trends, anomalies, and recommendations.

- Diagnosis Agent
  The Diagnosis Agent performs hybrid reasoning for machinery troubleshooting through five steps:

i. Problem Understanding: An LLM extracts structured symptom information including affected components, relevant metrics to check, severity estimate, problem category (engine, fuel system, cooling, propulsion, etc.), and temporal context (sudden vs. gradual).
ii. Data Analysis: Recent NoonReport data is fetched for the identified vessel, and the analytics engine computes trends, averages, and statistical anomaly detection to identify data-backed observations.
iii. Knowledge Retrieval: The RAG sub-pipeline searches the Pinecone vector store for relevant troubleshooting procedures, maintenance guides, and SOPs from uploaded manuals, followed by cross-encoder reranking.
iv. Hybrid Reasoning: The LLM combines data-driven observations with knowledge-based recommendations to produce a root cause analysis that is both empirically grounded and procedurally informed. 5. Structured Response: The output includes severity assessment, identified symptoms, probable causes ranked by likelihood, actionable recommendations, and supporting data observations.

C. *Document Ingestion Pipeline*

Users upload PDF documents through the frontend, triggering a background processing pipeline: text extraction via PyPDF, page-to-character offset mapping, intelligent chunking using LangChain's RecursiveCharacterTextSplitter (chunk size: 800 characters, overlap: 200 characters, with hierarchical separators: paragraph, line, sentence, word), embedding generation via SentenceTransformer, and batch upsert to Pinecone (batches of 100 vectors with SHA-256 hashed IDs). Document status transitions from "pending" to "completed" are tracked, and the frontend polls for processing status.

D. *Noon Report Data Management*

Operational data enters the system through CSV/Excel file imports. The import pipeline features flexible column normalization (mapping 100+ column name aliases to canonical field names), multi-format date/time parsing, decimal range validation against marine engineering constraints, duplicate detection by vessel-date-time composite key, and partial success support where valid rows are saved even when some rows fail validation. Import progress and per-row error tracking are maintained throughout processing.

E. *Administrative Module*

The administration module provides role-based access control with four user roles (admin, engineer, analyst, viewer), configurable guardrails rules, runtime-editable system configuration via key-value pairs, immutable audit logging for administrative actions, centralized system logging (query events, ingestion events, errors, security),

and internet search usage tracking. A custom Django admin dashboard provides real-time system health monitoring.

F. *Alert Engine & Real-Time Notifications*

MarineMind includes a rule-based alert engine that automatically monitors vessel operational data and system events, generating alerts when anomalous or critical conditions are detected. The alert system operates across two categories performance alerts derived from noon report data analysis, and system alerts triggered by platform events such as ingestion failures or high-severity diagnoses.

- Performance Alert Rules
  The alert engine evaluates the following operational rules each time a new NoonReport is submitted: -

i. Fuel Consumption Spike:
Triggers when FO consumption exceeds 1.5× the vessel's 30-day rolling average, escalating from a warning at 1.5× to an error at 2× or above. This catches sudden fuel inefficiencies that may indicate engine fouling, hull resistance changes, or faulty fuel metering.

ii. Low Fuel ROB (Remaining on Board):
Issues a warning when remaining fuel falls below 50 MT and a critical alert below 20 MT, ensuring timely bunkering decisions before fuel reserves reach unsafe levels.

iii. Speed Degradation:
Fires when actual speed drops below 85% of the ordered speed (warning), escalating to an error below 75%. This detects propulsion system issues, adverse weather impact, or hull fouling that reduce vessel performance.

iv. High Exhaust Temperature:
Generates a warning when main engine exhaust temperature exceeds 400°C and a critical alert above 450°C, which may indicate combustion irregularities, injector failures, or turbocharger issues.

v. Propeller Slip Anomaly:
Triggers a warning when calculated propeller slip exceeds 15%, escalating to an error beyond 25%. Excessive slip may indicate hull or propeller fouling, blade damage, or shaft alignment problems.

vi. SFOC (Specific Fuel Oil Consumption) Spike:
Issues a warning when SFOC exceeds 1.5× the 30-day rolling average, signaling declining engine thermal efficiency.

Each rule compares incoming noon report values against either fixed marine engineering thresholds (e.g., exhaust temperature limits) or statistically derived baselines (rolling averages). The 30-day rolling average approach adapts to vessel-specific operating profiles, avoiding false positives from fleet-wide fixed thresholds. Alerts include contextual detail — for example, a fuel spike alert reports both the current value and the baseline average for immediate interpretability.

- System & Diagnostic Alert Rules
  Beyond performance monitoring, the alert engine responds to system-level events:

i. Document Ingestion Failure: When a PDF processing pipeline fails during text extraction, chunking, or vector upsert, an error-level alert is generated with the failure reason, enabling administrators to investigate and re-trigger ingestion.

ii. Noon Report Import Issues:Partial or complete failures during CSV/Excel import generate alerts with row-level error details, distinguishing between validation errors (data quality) and system errors.

iii. High-Severity Diagnosis: When the Diagnosis Agent identifies a critical or high-severity machinery issue, the alert engine automatically creates a corresponding alert, ensuring that diagnostic findings are surfaced to fleet operators even if they are not actively monitoring the chat interface.

iv. Query Pipeline Errors: Unhandled exceptions in the chatbot query processing pipeline generate error alerts, providing observability into system reliability.

- Alert Deduplication
  To prevent alert fatigue, the engine employs deduplication based on alert type, associated vessel, and a configurable time window (default: 1 hour). If an identical alert (same type, vessel, and severity) was generated within the deduplication window, a new alert is suppressed. This is particularly important for performance alerts, where multiple noon reports in quick succession (e.g., during bulk imports) could otherwise generate redundant notifications.

- Frontend Notification System
  The frontend implements a polling-based notification system that checks for unread alerts every 30 seconds. New alerts are displayed as toast notifications in the top-right corner of the interface, color-coded by severity (blue for info, amber for warning, red for error, dark red for critical). Each toast auto-dismisses after 8 seconds but can be manually dismissed. The notification badge on the dashboard provides a persistent count of unread alerts, and the full alerts table on the DashboardPage supports filtering by type and severity, bulk mark-as-read, and historical review. This approach ensures that critical operational conditions are surfaced to users regardless of which page they are currently viewing.

V. Evaluation

A. *Evaluation Methodology*

The system was evaluated across four dimensions: retrieval quality, graph generation accuracy, diagnostic reasoning quality, and system performance. All tests used maritime-domain test cases derived from real vessel technical manuals and operational data.

B. *Retrieval Quality (RAG Agent)*

The RAG pipeline was evaluated on its ability to retrieve

and surface relevant information from uploaded maritime technical documents.

- Multi-Query Expansion Impact: Query expansion from a single rewritten query to 4–5 query variants increased the candidate chunk pool by approximately 2.5x while deduplication reduced redundancy, resulting in a broader yet non-redundant retrieval set. This addressed the vocabulary mismatch problem common in maritime documents where the same concept may be described with different terminology across manufacturers' manuals.

- Cross-Encoder Reranking Impact:Applying the cross-encoder reranker (ms-marco-MiniLM-L-6-v2) after initial embedding-based retrieval significantly improved precision. Chunks that scored highly on embedding similarity but were contextually irrelevant were filtered by the relevance threshold (0.1), while contextually rich chunks with lower embedding scores were promoted. The reranking step reduces the top-5 chunk set to highly relevant passages, improving the grounding quality of generated answers.

- Citation Accuracy: Generated responses were evaluated for citation faithfulness whether inline citations accurately corresponded to the source passages they referenced. The mandatory citation prompting strategy combined with citation map resolution ensured that users could verify every factual claim against its source document and page number.

C. *Graph Generation Accuracy (Graph Agent)*

The Graph Agent was evaluated on its ability to correctly interpret natural language queries and produce accurate visualizations.

- Intent Parsing: Across test queries spanning fuel trends, speed-consumption relationships, RPM analysis, weather impact, fleet comparisons, and anomaly detection, the LLM-based intent parser correctly extracted the graph type, target metric, and vessel reference. Fuzzy vessel name matching handled common abbreviations and partial names.

- Data Accuracy: Generated charts were verified against direct database queries to confirm that the underlying data (labels, values, aggregations) was mathematically correct. The analytics engine leverages Django ORM aggregation functions (Avg, Sum, StdDev, Trunc) which operate at the database level, ensuring computational accuracy.

- Visualization Correctness: Chart configurations were validated for correct Chart.js schema compliance, appropriate chart type selection (line for time-series, scatter for correlations, bar for categorical comparisons), accurate axis labeling, and meaningful title generation.

D. *Diagnostic Reasoning Quality (Diagnosis Agent)*

The Diagnosis Agent was evaluated on diagnostic completeness and actionability using simulated machinery fault scenarios.

- Symptom Extraction: The structured symptom extraction step was tested with varying levels of user description specificity — from brief complaints ("engine overheating") to detailed reports ("exhaust temperature rose from 350°C to 420°C over the past week with stable RPM at 85"). The LLM consistently extracted relevant symptoms, identified affected components, and selected appropriate NoonReport metrics for data analysis.

- Hybrid Reasoning Quality: The combination of data-driven observations (anomaly detection from NoonReport trends) with knowledge-based recommendations (retrieved from technical manuals) produced diagnoses that were more comprehensive than either approach alone. Data analysis identified statistical anomalies that corroborated or contradicted the reported symptoms, while manual knowledge provided procedural context for recommended actions.

- Actionability: Diagnostic outputs were evaluated for whether recommendations were specific, prioritized by likelihood, and actionable by marine engineers. The structured output format (severity, symptoms, causes, recommendations, data observations) provided a clear triage framework.

E. *System Performance*

- Response Latency: End-to-end response times were measured across agent types. Simple RAG queries completed in 5–15 seconds (dominated by LLM inference and vector search). Graph queries completed in 3–10 seconds (LLM intent parsing + database aggregation + chart configuration). Diagnosis queries, being the most complex (multiple LLM calls + data analysis + RAG sub-pipeline), completed in 15–60 seconds. All times include network latency to external APIs (HuggingFace, Pinecone, Tavily).

- Scalability: The document ingestion pipeline processes PDFs in background threads with batched vector upserts (100 vectors per batch). The chunking strategy (800-character chunks with 200-character overlap) balances retrieval granularity with context preservation. Pinecone's serverless architecture provides automatic scaling for vector search workloads.

- Safety and Robustness: The Guardrails Agent effectively filtered prompt injection attempts, off-topic queries, and harmful content while maintaining a liberal pass-through for legitimate technical queries. The zero-temperature LLM classification minimized stochastic variability in safety decisions.

VI. Conclusion

This paper introduces MarineMind, a multi-agent AI system that improves maritime knowledge management and decision-making by using specialized agents such as RAG, Internet Search, Graph Analytics, and Diagnosis instead of a single chatbot. The system enhances retrieval accuracy through multi-query expansion and reranking, enables natural language-driven data analysis via graph-based insights, and provides reliable troubleshooting by combining operational data with technical knowledge. Its modular Guardrails-Router-Executor architecture ensures scalability, flexibility, and

maintainability, while demonstrating how modern AI techniques can practically reduce information search time, support better decisions, and assist marine engineers in solving real-world problems efficiently.

VII. Future Work

Several directions for future enhancement are identified:

i. Real-Time Sensor Integration:

The current system relies on manually uploaded noon reports for operational data. Future work will integrate real-time sensor data streams from vessel IoT systems (AIS, engine monitoring systems, fuel flow meters) to enable continuous performance monitoring and proactive anomaly alerting without manual data entry.

ii. Predictive Maintenance Models:

Building on the Diagnosis Agent's data analysis capabilities, supervised machine learning models (e.g., LSTMs, transformer-based time-series models) can be trained on historical failure data to predict equipment failures before they occur, shifting from reactive diagnosis to predictive maintenance.

iii. Multi-Modal Document Understanding:

The current ingestion pipeline processes PDF text only. Future versions will incorporate vision-language models to extract information from engineering diagrams, P&ID (Piping and Instrumentation Diagrams), wiring schematics, and tabular data within scanned documents, significantly expanding the retrievable knowledge base.

iv. Fine-Tuned Maritime LLM:

The current system uses general-purpose LLMs via HuggingFace. Fine-tuning a domain-specific model on maritime corpora (IMO regulations, classification society rules, marine engineering textbooks) would improve the accuracy of intent parsing, routing, and generation while reducing hallucination in safety-critical maritime contexts.

v. Collaborative Features:

Adding multi-user annotation capabilities, shared diagnostic case histories, and fleet-wide knowledge sharing would enable organizational learning. Engineers could annotate diagnostic outcomes, validate AI recommendations, and build a feedback loop that continuously improves system accuracy.

vi. Regulatory Compliance Automation:

Integrating regulatory databases (IMO, MARPOL, SOLAS) and automating compliance checking against vessel operational data would help operators ensure ongoing compliance with evolving international maritime regulations.

vii. Offline-Capable Deployment:

For vessels with limited or intermittent internet connectivity, a lightweight on-device deployment with smaller quantized models and a local vector store would ensure AI assistance remains available at sea. Synchronization with the cloud-based system when connectivity is restored would maintain data consistency.

viii. Enhanced Evaluation Framework:

Developing a standardized maritime AI benchmark with annotated ground-truth datasets for retrieval, diagnosis, and analytics tasks would enable rigorous quantitative evaluation and comparison with future systems in the domain.

Acknowledgment

We would like to express our sincere gratitude to the Head of MTM Ship Management and their team for providing access to the ship machinery data and for their continuous cooperation throughout this project. Their support was invaluable in enabling us to conduct realistic testing and validation of our system. We also extend our heartfelt thanks to our guide, Prof. Nargis Shaikh, for her expert guidance, encouragement, and constructive feedback that significantly contributed to the successful completion of this work. Finally, we appreciate the support and insights from all colleagues and peers who provided valuable suggestions and encouragement during the research

References

[1] Kuhl, F., Smith, J., & Brown, L., “Deep Learning in Maritime Anomaly Detection: LSTM Autoencoders with Decision Trees,” *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 9, pp. 9876–9887, Sep. 2023.

[2] Chen, H., Li, Y., & Zhang, Q., “AIS-Based Outlier Detection Using Bi- GRU Encoder–Decoder Models,” *IEEE Access*, vol. 11, pp. 34567– 34578, 2023.

[3]

[4] S. Patel et al., "Predictive Maintenance in Maritime Industry: A Machine Learning Approach," *IEEE Access*, vol. 13, pp. 5678–5689, Jan. 2025.

[5] J. Lee and H. Kim, "Integration of IoT and AI for Smart Ship Systems," *IEEE Internet Things J.*, vol. 12, no. 3, pp. 789–800, Mar. 2025.

[6] M. Gupta and R. Verma, "Development of a Mobile Application for Onboard Quick Access to Ship Manuals," *IEEE Trans. Mobile Comput.*, vol. 24, no. 4, pp. 1122–1133, Apr. 2025.

[7] D. Simion et al., "AI-Driven Predictive Maintenance in Modern Maritime Operations," *MDPI Electronics*, vol. 14, no. 20, pp. 9439–9450, Oct. 2024.

[8] Samsung Electronics, "Samsung Unveils AI IoT Technology for Ships," *Riviera Maritime Media*, Jan. 15, 2025.